

ЛАБОРАТОРНАЯ РАБОТА № 11

Классы представлений

Цель: изучить использование классов представлений и постраничной навигации.

Содержание отчета: титульный лист, цель работы, задание, описание хода выполнения задания со скриншотами, листинги, вывод.

Введение в CBV (Class Based Views). Класс View

В предыдущих лабораторных работах мы использовали функции представления. Это довольно простой подход и используется, в основном, когда нужно реализовать простую логику обработки запросов. Однако, во фреймворке Django, помимо функций, можно использовать и классы представлений (представления, основанные на классах):

CBV – Class-Based Views

<https://docs.djangoproject.com/en/4.2/ref/class-based-views/>

Чаще всего, именно их применяют на практике, так как объектно-ориентированный подход, зачастую, позволяет заметно упростить код, сделать его более понятным и читабельным.

Все классы представлений в Django наследуются от базового класса **View**, а дочерние специализируются на различных задачах обработки и формирования результата. Все их мы последовательно разберем, а начнем с базового класса **View**.

Если перейти по ссылке:

<https://docs.djangoproject.com/en/4.2/topics/class-based-views/intro/>

то можно подробно почитать о порядке использования классов представлений, в том числе и класса **View**. Он поддерживает два метода **get()** и **post()** для обработки GET и POST-запросов соответственно. Давайте начнем с внедрения класса **View** в наш проект.

У нас имеется подходящая функция представления **addpage()**, которая содержит обработку GET и POST-запросов. Определим после нее класс представления с именем **AddPage**, унаследованный от класса **View**:

```
class AddPage(View):
    def get(self, request):
        pass
```

```
def post(self, request):  
    pass
```

В этом классе мы объявили два метода **get()** и **post()**, которые будут срабатывать при GET и POST-запросах соответственно. Каждый метод должен иметь дополнительный параметр **request** с данными запроса. Сам же базовый класс **View** необходимо импортировать из следующей ветки:

```
from django.views import View
```

Если перейти внутрь этого класса, то можно увидеть список из всех поддерживаемых им методов. А также специальный метод **as_view()**, который используется для привязки этого класса к маршрутам URL-адресов.

Но для начала мы определим реализации методов **get()** и **post()**. Для этого достаточно скопировать соответствующие строки из функции **addpage()**. Получим следующую реализацию класса:

```
class AddPage(View):  
    def get(self, request):  
        form = AddPostForm()  
        return render(request, 'women/addpage.html',  
{ 'menu': menu, 'title': 'Добавление статьи', 'form':  
form })  
  
    def post(self, request):  
        form = AddPostForm(request.POST, request.FILES)  
        if form.is_valid():  
            form.save()  
            return redirect('home')  
  
        return render(request, 'women/addpage.html',  
{ 'menu': menu, 'title': 'Добавление статьи', 'form':  
form })
```

То есть в методе **get()** мы просто отдаем пустую форму, а в методе **post()** создаем форму с принятыми данными, и, если данные корректны, то сохраняем их в БД и делаем перенаправление на главную страницу, иначе, возвращаем форму с ранее введенными значениями и списком ошибок ее заполнения.

Всю логику по вызову нужных методов (**get**, **post** и др.) базовый класс **View** берет на себя. Нам остается только разнести код по этим методам и связать класс **AddPage** с маршрутом добавления новой статьи. Для этого перейдем в файл **women/urls.py** и в списке **urlpatterns** для маршрута **addpage** пропишем класс **AddPage**:

```
urlpatterns = [
    path('', views.index, name='home'),
    path('about/', views.about, name='about'),
    path('addpage/', views.AddPage.as_view(),
name='add_page'),
    ...
]
```

Мы заменили функцию представления **addpage()** на класс представления **AddPage**. После запуска тестового веб-сервера и перехода на страницу:

<http://127.0.0.1:8000/addpage/>

видим срабатывание метода **get()** и отображение пустой формы для добавления статьи в окне браузера. Если заполнить эту форму и выполнить отправку данных, то мы попадаем во второй метод **post()**, где эти данные после проверки сохраняются в БД.

Вот минимальный функционал, который предоставляет нам базовый класс **View**. По сути, он позволяет разнести код программы по отдельным методам, что гораздо удобнее описания всей логики в пределах одной функции.

Класс **TemplateView**

Рассмотрим пример использования еще одного класса представления **TemplateView**. Он служит для обработки шаблонов и отправки результата пользователю. Например, у нас есть функция **index()**, которая возвращает список постов, используя шаблон **index.html**. Заменим эту функцию классом **TemplateView**. Для этого мы определим новый класс **WomenHome** следующим образом:

```
class WomenHome(TemplateView):
    template_name = 'women/index.html'
```

Дополнительно должны импортировать базовый класс из ветки:

```
from django.views.generic import TemplateView
```

Обратите внимание на специальный атрибут **template_name**. Он служит для указания шаблона, который будет использоваться при обращении к данному классу.

Давайте в таком виде свяжем маршрут главной страницы с классом **WomenHome** и посмотрим, что получится (в файле **women/urls.py**):

```
urlpatterns = [
    path('', views.WomenHome.as_view(), name='home'),
    ...
]
```

Видим отображение пустого шаблона без списка статей и пунктов меню. Почему так произошло? Очевидно, по той причине, что мы в шаблон не передаем никаких дополнительных данных. Как выполнить эту передачу? Если перейти в базовый класс **TemplateView**, то увидим здесь реализацию метода **get()**, в котором формируется контекст (набор данных) с помощью метода **get_context_data()** с последующей передачей результата в шаблон. Причем этот метод определен в классе миксина **ContextMixin**. И в нем мы видим дополнительный атрибут **extra_context**. Само название атрибута говорит о возможности определения через него дополнительного контекста (дополнительных данных).

Возвратимся в класс **WomenHome** и определим в нем этот атрибут **extra_context** следующим образом:

```
class WomenHome(TemplateView):
    template_name = 'women/index.html'
    extra_context = {
        'title': 'Главная страница',
        'menu': menu,
        'posts':
Women.published.all().select_related('cat'),
        'cat_selected': 0,
    }
```

Здесь просто скопировано определение словаря **data** из функции **index**. После обновления главной страницы увидим ее в прежнем виде.

Также в методе **as_view()** мы можем передавать дополнительные аргументы для классов представлений. Например, используя именованный параметр **extra_context**, можно передавать в класс любые статические данные, которые будут доступны в шаблоне. По сути, все данные из **extra_context** будут автоматически присвоены одноименному атрибуту и переданы в шаблон. Поэтому строка:

```
WomenHome.as_view(extra_context={'title': "Главная
страница сайта"})
```

передает в шаблон параметр **title** с указанным значением. Но мы это уберём (это было сделано исключительно в учебных целях), оставим, как было.

Обратите внимание, что словарь **extra_context** можно использовать именно для заранее сформированных данных, которые затем уже никак не меняются. Если же мы собираемся передавать данные, которые формируются динамически в момент поступления запроса, например, менять параметр **cat_selected** в зависимости от параметра **cat_id** GET-запроса:

http://127.0.0.1:8000/category/?cat_id=2

то через атрибут **extra_context** сделать это уже не получится. Для этого потребуется переопределять метод **get_context_data** базового класса. В принципе, с помощью метода **get_context_data()** можно передавать и статические и динамические данные, то есть любую информацию в шаблон. Поэтому нередко можно встретить использование этого метода. В нашем примере им можно воспользоваться так:

```
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context['title'] = 'Главная страница'
    context['menu'] = menu
    context['posts'] =
Women.published.all().select_related('cat')
    context['cat_selected'] =
int(self.request.GET.get('cat_id', 0))
    return context
```

Если теперь попробовать поменять значения **cat_id** в GET-запросе, то будем видеть подсветку различных разделов. Правда, само содержимое будет оставаться прежним, т.к. мы этот функционал здесь не прописывали. Мы лишь показали таким образом отличия между методом **get_context_data()** и атрибутом **extra_context**.

Вот так можно использовать базовый класс представления **View** и его специализированный дочерний класс **TemplateView**. Конечно, в реальной практике эти классы применяются не часто, так как они описывают достаточно общий функционал.

Класс **ListView**

Ранее мы заменили функцию **index()** на класс представления **WomenHome**, который наследовался от класса **TemplateView**. Однако можно сделать еще лучше и использовать другой базовый класс **ListView**, созданный специально для отображения произвольных списков:

<https://docs.djangoproject.com/en/4.2/ref/class-based-views/>

У нас на главной странице отображается список статей. Сначала импортируем класс **ListView**:

```
from django.views.generic import ListView
```

И укажем его в качестве базового для класса **WomenHome**:

```
class WomenHome(ListView):  
    ...
```

Если сейчас запустить тестовый веб-сервер и перейти на главную страницу, то увидим ошибку. Она связана с тем, что класс **ListView** предполагает получение данных из таблиц БД. И для этого (в самом простом варианте) в классе **WomenHome** нужно определить специальный атрибут:

```
class WomenHome(ListView):  
    model = Women
```

Все остальные строки класса **WomenHome** закомментируем.

Фактически, строка **model = Women** выбирает все записи из таблицы **women** и пытается отобразить их в виде списка, используя шаблон с именем:

<имя приложения>/<имя модели>_list.html

Именно эту ошибку «TemplateDoesNotExist» мы видим при обновлении главной страницы, так как фреймворк не находит шаблон по умолчанию:

women/women_list.html

Конечно, мы могли бы создать такой шаблон, но у нас уже есть свой собственный для этих целей – **women/index.html**. Чтобы его указать в классе представлений, используется атрибут **template_name**, которому присваиваем путь к нужному шаблону:

```
class WomenHome(ListView):  
    model = Women  
    template_name = 'women/index.html'
```

Если теперь обновить главную страницу, то увидим пустой список так, словно у нас нет ни одной статьи. Почему это произошло? Дело в том, что мы в шаблоне обращаемся к переменным со своими именами, которые определили в функции представления **index**. Например, **post** содержал список всех записей. Что же теперь нужно прописать вместо **post**? По умолчанию, данные из модели **Women**, указанной в классе представлений, помещаются в коллекцию **object_list**. Если мы ее укажем вместо **post**, то все заработает. Обновляя главную страницу, видим список всех постов.

Если в шаблоне вместо **object_list** мы хотим использовать другое обозначение (имя), то в классе **WomenHome** следует прописать атрибут **context_object_name** с указанием другого имени переменной:

```
context_object_name = 'posts'
```

И, далее, в шаблоне **index.html** снова можем писать **posts**.

Осталось в **index.html** передать остальные данные. Для этого воспользуемся атрибутом **extra_context**:

```
class WomenHome(ListView):
    model = Women
    template_name = 'women/index.html'
    context_object_name = 'posts'

    extra_context = {
        'title': 'Главная страница',
        'menu': menu,
#         'posts':
Women.published.all().select_related('cat'),
        'cat_selected': 0,
    }
```

С помощью атрибута **extra_context** мы можем передавать в шаблон только те данные, которые можно сформировать в момент определения самого класса. Если же данные зависят от параметров GET-запроса, то для этого следует переопределять метод **get_context_data()**. Для класса **ListView** он имеет следующий вид:

```
class WomenHome(ListView):
    model = Women
    template_name = 'women/index.html'
    context_object_name = 'posts'

    def get_context_data(self, *, object_list=None,
**kwargs):
        context = super().get_context_data(**kwargs)
        context['title'] = 'Главная страница'
        context['menu'] = menu
        context['cat_selected'] = 0
        return context
```

Здесь дополнительно определен параметр **object_list**.

Главная страница приобрела почти тот же самый вид. Почему почти? Потому что строчка **model = Women** выбирает все записи из таблицы **women**, а мы это делали с помощью команды:

```
Women.published.all().select_related('cat')
```

Чтобы это поправить, вместо атрибута **model** следует переопределить специальный метод **get_queryset()** следующим образом (атрибут **model** следует удалить из класса):

```
def get_queryset(self):  
    return  
Women.published.all().select_related('cat')
```

Вот так, переопределяя метод **get_queryset()**, можно задавать свой алгоритм выборки записей из модели.

На первый взгляд, может показаться, что мы особо не сократили программный код. Это действительно так. Объясняется очень просто. У нас была достаточно простая реализация в функции представления **index()**, и, как следствие, не имеем ничего нового в классе представления **WomenHome**. Но дальше мы увидим, как, например, пагинация (разбивка на страницы) может быть легко реализована в классе буквально несколькими дополнительными атрибутами. Вообще, преимущества классов представлений перед функциями все больше и больше проявляются с ростом сложности проекта. Это происходит за счет большей модульности программного кода (благодаря классам и механизму наследования) и вынесения типовых решений в базовые классы.

Создаем класс представлений для категорий

Мы создали класс представления для главной страницы. Повторим этот процесс и пропишем аналогичный класс для отдельных категорий. Сначала объявим класс **WomenCategory** с тем же базовым классом **ListView**, указав те же самые атрибуты и методы:

```
class WomenCategory(ListView):  
    template_name = 'women/index.html'  
    context_object_name = 'posts'  
  
    def get_context_data(self, *, object_list=None,  
**kwargs):  
        context = super().get_context_data(**kwargs)  
        cat = context['posts'][0].cat  
        context['title'] = 'Категория - ' + cat.name  
        context['menu'] = menu  
        context['cat_selected'] = cat.id
```



```

        return context

    def get_queryset(self):
        return
Women.published.filter(cat__slug=self.kwargs['cat_slug']
    ).select_related('cat')

```

Обратите внимание, как записана выборка записей. Здесь первым параметром указано имя **cat__slug** – это способ обращения к слягу таблицы **category** через объект **cat** модели **Women**. Далее указываем, что поле **slug** у категории должно быть равно параметру **cat_slug**, который мы берем из словаря **kwargs** объекта класса **WomenCategory**. Ключ **cat_slug** автоматически формируется по шаблону маршрута (файл **women/urls.py**), в котором мы должны вместо функции указать класс представления:

```

path('category/<slug:cat_slug>/',
WomenCategory.as_view(), name='category'),

```

Если теперь перейти на страницу сайта и выбирать категории, то будут отображаться статьи выбранной рубрики. Но, если указать несуществующий слаг, то увидим пустую страницу, а нам бы хотелось увидеть ошибку 404 – страница не найдена. Для этого в классе **WomenCategory** запишем атрибут:

```

allow_empty = False

```

который указывает генерировать исключение 404, если список статей пуст. Так мы сохраняем общий функционал нашего сайта.

Класс **DetailView**

Заменим функцию **show_tag_postlist()** классом представления с именем **TagPostList**, унаследованным от базового класса **ListView**, для отображения постов по выбранному тегу:

```

class TagPostList(ListView):
    template_name = 'women/index.html'
    context_object_name = 'posts'
    allow_empty = False

    def get_context_data(self, *, object_list=None,
**kwargs):
        context = super().get_context_data(**kwargs)
        tag =
TagPost.objects.get(slug=self.kwargs['tag_slug'])
        context['title'] = 'Ter: ' + tag.tag
        context['menu'] = menu
        context['cat_selected'] = None

```

```

        return context

    def get_queryset(self):
        return
Women.published.filter(tags__slug=self.kwargs['tag_slug']
    ]).select_related('cat')

```

Создадим еще один класс для отображения отдельных постов и заменим функцию представления **show_post()**. Для этого хорошо подходит класс **DetailView**. Объявим класс **ShowPost** после функции **show_post()** и унаследуем его от **DetailView**. В самом простом варианте его можно записать следующим образом:

```

class ShowPost(DetailView):
    model = Women
    template_name = 'women/post.html'

```

Мы здесь сразу указали два атрибута: **model** – для модели; **template_name** – для используемого шаблона.

Пока ничего прописывать в нем не будем, а перейдем к списку маршрутов (**women/urls.py**). Вместо строки:

```
path('post/<slug:post_slug>/', views.show_post, name='post'),
```

запишем:

```

path('post/<slug:post_slug>/',
views.ShowPost.as_view(), name='post'),

```

Теперь при попытке просмотра какого-либо поста, возникает исключение «**AttributeError**». В чем проблема? Класс **DetailView** по умолчанию пытается выбрать из указанной модели **Women** запись, используя атрибут **pk** или **slug**. Но у нас формируется маршрут с параметром **post_slug**, из-за этого и возникает такая ошибка.

В самом простом случае мы можем в шаблоне маршрута вместо **post_slug** записать просто **slug**, и тогда ошибки уже не будет. Или же в классе **ShowPost** прописать атрибут:

```
slug_url_kwarg = 'post_slug'
```

Если используется идентификатор, то прописывается атрибут **pk_url_kwarg**. Обычно эти атрибуты опускают и в параметрах маршрутов используют ключевые слова **slug** – для слага и **pk** – для идентификаторов.

При попытке вывести статью отображается пустая страница. Это происходит из-за использования параметра **post** внутри шаблона **post.html**. По умолчанию класс **DetailView** в шаблон передает переменную с именем **object** и переменную с именем модели (малыми буквами). В нашем примере – это **women**.

Чтобы указать другое имя переменной для шаблона, мы в классе **ShowPost** должны прописать атрибут:

```
context_object_name = 'post'
```

Тогда при обновлении страницы видим корректное содержимое поста. А, если указать несуществующий слаг, то автоматически будет сгенерировано исключение 404 – страница не найдена.

Осталось передать в шаблон заголовок **title** и пункты главного меню:

```
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context['title'] = context['post']
    context['menu'] = menu
    return context
```

Сейчас происходит отображение статьи по слагу вне зависимости от статуса ее публикации (она может иметь флаг **is_published=False**). Исправим это. В классе **ShowPost** нужно переопределить метод **get_object()**, который отвечает за извлечение записи:

```
def get_object(self, queryset=None):
    return get_object_or_404(Women.published,
slug=self.kwargs[self.slug_url_kwarg])
```

Мы здесь используем функцию **get_object_or_404()**, которая возвращает одну запись из таблицы **Women** менеджера **published**, если она будет найдена, либо сгенерирует ошибку 404 – страница не найдена.

Обратите внимание, что в функции **get_object_or_404()** первым аргументом мы передаем менеджер **published**. Так тоже можно делать, и это гораздо удобнее, чем прописывать отдельно дополнительное условие **is_published=True**. Мы его можем не помнить, для этого и был создан отдельный менеджер **published**.

Теперь опубликованные статьи успешно отображаются в браузере.

Класс **FormView**

Рассмотрим следующий класс представлений – класс **FormView**. Подробную информацию о нем можно посмотреть по следующей ссылке:

<https://docs.djangoproject.com/en/4.2/ref/class-based-views/generic-editing/#formview>

Он предназначен для автоматизации отображения и обработки HTML-форм. Посмотрим, как его можно применить в нашем проекте.

Ранее мы создали класс **AddPage** для отображения формы добавления статей. Но наследовали его от базового класса **View**. Правильнее было бы здесь использовать класс **FormView**. Для этого еще раз сделаем объявление этого класса со следующими атрибутами:

```
class AddPage(FormView):  
    form_class = AddPostForm  
    template_name = 'women/addpage.html'  
    success_url = reverse('home')
```

Сам базовый класс расположен в ветке:

```
from django.views.generic.edit import FormView
```

Названия атрибутов класса говорят сами за себя. Первый **form_class** должен ссылаться на класс формы. Второй **template_name** – на маршрут к шаблону для отображения формы. Наконец, третий **success_url** определяет маршрут перенаправления при успешной отправке после валидации формы. Однако, если сейчас попытаться запустить веб-сервер, то увидим сообщения об ошибках. Они связаны с использованием функции **reverse()**. Она пытается построить маршрут по имени 'home', но этот маршрут на момент формирования класса **AddPage** не определен. В таких ситуациях следует пользоваться другой аналогичной функцией:

```
success_url = reverse_lazy('home')
```

которая также строит маршрут по имени 'home', но не сразу при формировании класса, а в момент, когда он реально понадобится фреймворку Django. Здесь суффикс **_lazy** означает «ленивый», то есть маршрут выстраивается лишь в момент его использования, а не заранее. В данном случае это очень важно.

Вообще, всегда вместо функции **reverse()** можно прописывать функцию **reverse_lazy()**. Тогда ошибок, связанных с отложенным формированием маршрута, возникать не должно. Поэтому на практике довольно часто можно увидеть вызов именно **reverse_lazy()**.

Итак, наш класс в самом простом варианте готов. Если сейчас перейти на страницу добавления статьи:

<http://127.0.0.1:8000/addpage/>

то заметим, что не хватает главного меню и заголовка страницы, так как мы в шаблон не передавали дополнительные параметры. Воспользуемся атрибутом **extra_context**:

```
class AddPage(FormView):
    form_class = AddPostForm
    template_name = 'women/addpage.html'
    success_url = reverse_lazy('home')
    extra_context = {
        'menu': menu,
        'title': 'Добавление статьи',
    }
```

Теперь страница с формой отображается, как и прежде. Попробуем добавить новый пост с произвольными данными, нажимаем кнопку «Добавить» и попадаем на главную страницу. Визуально все отработало, как и задумывалось, но в реальности введенные данные в БД добавлены не были. В этом легко убедиться во вкладке админ-панели со списком статей:

<http://127.0.0.1:8000/admin/women/women/>

Он не изменился. В действительности, мы лишь реализовали отображение формы с проверкой корректности введенных данных. Например, если ввести неверный слог, то увидим сообщение от модели формы. То есть сейчас все работает, кроме сохранения данных. Этот функционал мы должны прописать дополнительно.

Для этого в классе **AddPage** нужно переопределить специальный метод **form_valid()**. Например, так:

```
def form_valid(self, form):
    form.save()
    return super().form_valid(form)
```

Этот метод вызывается только после успешной проверки всех переданных данных формы. Параметр **form** – это ссылка на заполненную форму. Поэтому нам остается только вызвать метод **save()** и сохранить данные в БД. В конце обязательно нужно записать оператор **return** с вызовом этого же метода из базового класса.

Так как метод **form_valid()** вызывается после проверки данных, то в нем доступен словарь **form.cleaned_data**, который содержит всю корректную переданную информацию, и мы совершенно безопасно здесь можем ее использовать.

Снова заполним форму данными, нажимаем «Добавить» и видим в списке новую запись. Мы полностью повторили функционал предыдущего класса **AddPage**. Здесь только нужно учитывать, что объект формы в шаблон **addpage.html** передается через параметр **form**. Мы именно его использовали. Если указать любое другое имя, то форма отображаться не будет.

Класс **CreateView**

Если мы посмотрим на список классов представлений:

<https://docs.djangoproject.com/en/4.2/ref/class-based-views/>

то увидим, что после **FormView** следуют классы **CreateView**, **UpdateView** и **DeleteView**. По их названиям легко догадаться, что они еще более специализированы, чем класс **FormView**. В частности, для добавления (то есть создания) новой статьи мы могли бы вместо **FormView** использовать класс **CreateView**. Давайте это сделаем.

Уберем из класса **AddPage** метод **form_valid** и унаследуем класс от **CreateView**, получим:

```
class AddPage(CreateView):
    form_class = AddPostForm
    template_name = 'women/addpage.html'
    success_url = reverse_lazy('home')
    extra_context = {
        'menu': menu,
        'title': 'Добавление статьи',
    }
```

После запуска тестового веб-сервера, видим тот же самый результат. Причем мы можем удалить атрибут **success_url**, тогда маршрут перенаправления будет определяться на основе метода **get_absolute_url()** модели **Women**. Класс **CreateView** обращается к этому методу для получения URL-адреса добавленной статьи и перенаправляет нас к ней.

Таким образом, класс **CreateView** берет дополнительно на себя весь функционал по сохранению данных формы в БД. Причем модель была взята из класса **AddPostForm**, так как это форма, связанная с моделью. При необходимости, мы можем использовать класс **CreateView** с любой другой

моделью без указания формы. Для этого следует в нем определить атрибут **model** следующим образом:

```
class AddPage(CreateView):
    model = Women
    # form_class = AddPostForm
    template_name = 'women/addpage.html'
    success_url = reverse_lazy('home')
    extra_context = {
        'menu': menu,
        'title': 'Добавление статьи',
    }
```

Атрибут **form_class**, конечно, нужно убрать. Если в таком виде попытаться отобразить страницу добавления поста, то увидим ошибку, что не определена коллекция **fields** для указания отображаемых на форме полей. Пропишем этот атрибут в виде:

```
fields = '__all__'
```

В результате, будут отображены все поля формы с названиями, указанными в параметре **verbose_name** модели **Women**.

Если же требуется строго определенный набор полей, то вместо строки **'__all__'** следует прописать список с названиями полей из модели **Women**. Например, так:

```
fields = ['title', 'slug', 'content', 'is_published',
         'cat']
```

Обновляем страницу и видим только указанные поля формы. При этом весь функционал по проверке данных и добавлению статей остается неизменным.

Конечно, в этом списке должны быть перечислены все обязательные для заполнения поля. Например, если мы уберем поле **'cat'**:

```
fields = ['title', 'slug', 'content', 'is_published']
```

то при добавлении возникнет ошибка, что это поле не указано. Поэтому все, что нужно пользователю заполнить, следует отображать в форме. Либо делать автозаполнение на программном уровне.

Вообще, какой вариант использовать для класса **CreateView** решает сам программист, исходя из логики и удобства написания программного кода. Мы здесь показываем возможности отдельных классов представлений,

принципы их работы. А конкретная реализация должна вытекать из понимания работы этих классов и общей архитектуры создаваемого проекта.

Класс **UpdateView**

Следующий класс, который мы рассмотрим, это **UpdateView**. Он служит для изменения существующих записей. На данный момент у нас такого функционала нет. Поэтому давайте его добавим.

Объявим класс **UpdatePage** (после класса **AddPage**) и унаследуем его от базового класса **UpdateView**:

```
class UpdatePage(UpdateView):
    model = Women
    fields = ['title', 'content', 'photo',
              'is_published', 'cat']
    template_name = 'women/addpage.html'
    success_url = reverse_lazy('home')
    extra_context = {
        'menu': menu,
        'title': 'Редактирование статьи',
    }
```

Остальное содержимое мы оставили практически таким же, как и в классе **AddPage**, только коллекцию **fields** немного изменили (убрали поле **slug** и добавили поле **photo**).

Осталось связать этот класс с маршрутом. Перейдем в файл **women/urls.py** и в коллекции **urlpatterns** добавим строку:

```
urlpatterns = [
    ...
    path('edit/<int:pk>/', views.UpdatePage.as_view(),
         name='edit_page'),
    ...
]
```

Обратите внимание на параметр **pk** в этом маршруте. Это специальное имя (сокращение от слов **primary key** – главный ключ), по которому автоматически будет отбираться запись для редактирования в классе **UpdatePage**. Запись будет выбираться по ее идентификатору (полю **id**).

Посмотрим, как это будет работать. Перейдем, например, по адресу:

<http://127.0.0.1:8000/edit/1/>

и видим отображение формы с заполненными полями записи с идентификатором 1. Мы можем поменять здесь какие-либо данные, и, после нажатия на кнопку «Добавить», запись будет изменена в БД.

Если мы хотим отбирать записи не по **id**, а по слаг, то в маршруте вместо параметра **pk** следует прописать **slug** с соответствующим конвертером:

```
path('edit/<slug:slug>/', views.UpdatePage.as_view(),
name='edit_page'),
```

Теперь статьи автоматически выбираются по полю **slug**. Соответственно, в модели это поле так и должно называться – **slug**.

Класс **DeleteView**

Последний подобный класс **DeleteView** разберите самостоятельно. Используется он аналогично ранее рассмотренным классам **CreateView** и **UpdateView**. Подробно о нем написано в документации:

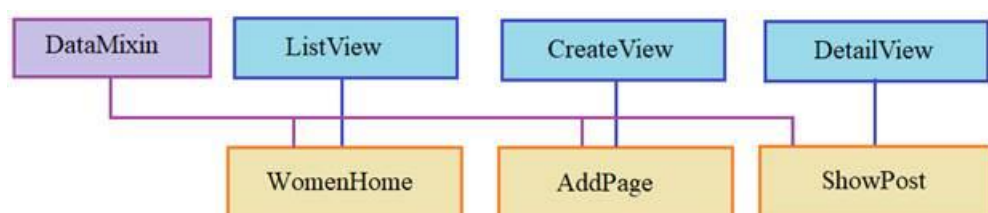
<https://docs.djangoproject.com/en/4.2/ref/class-based-views/generic-editing/#deleteview>

Mixins как способ улучшения программного кода

Вынесем общий код классов представлений в отдельный класс, который можно воспринимать как **миксин** (mixin). Идею этого паттерна поясним на простом примере.

Предположим, у нас имеется класс **Furniture** (мебель). Тогда мы можем определить его функциональность в зависимости от набора базовых классов – миксинов. Например, ножки (**LegsMixin**) и квадратная крышка (**CoverRectMixin**) будут определять квадратный стол; ножки и круглая крышка (**CoverRoundMixin**) определяют круглый стол; сочетание ножек, квадратной крышки и спинки (**BackMixin**) дают стул. Это наглядная идея классов миксинов, которые своими различными сочетаниями описывают функционал дочерних классов.

В разных языках программирования миксины реализуются по-разному. В частности, в Python, благодаря наличию механизма множественного наследования, примеси можно добавлять в виде отдельного базового класса:



Например, наши классы представлений можно дополнительно унаследовать от класса **DataMixin**, который будет отвечать за наполнение шаблонов стандартной информацией. Благодаря этому, мы сможем сократить дублирование кода в тексте программы.

Начнем с класса **WomenHome**. Классы миксинов принято записывать первыми в списке наследования, поэтому получим такую запись:

```
class WomenHome(DataMixin, ListView):  
    ...
```

Это объясняется тем, что в Python класс, записанный первым, первым и обрабатывается. Поэтому данные базового класса **DataMixin** не будут переопределять какие-либо атрибуты следующего класса **ListView**.

Теперь объявим новый класс **DataMixin**. Где это лучше сделать? Обычно в Django все дополнительные, вспомогательные классы определяют в отдельном файле **utils.py** текущего приложения. Мы так и поступим. Создадим этот файл и в нем запишем класс **DataMixin** следующим образом:

```
menu = [{ 'title': "О сайте", 'url_name': 'about'},  
         { 'title': "Добавить статью", 'url_name':  
'add_page'},  
         { 'title': "Обратная связь", 'url_name':  
'contact'},  
         { 'title': "Войти", 'url_name': 'login'}  
]  
  
class DataMixin:  
    def get_mixin_context(self, context, **kwargs):  
        context['menu'] = menu  
        context['cat_selected'] = None  
        context.update(kwargs)  
        return context
```

Обратите внимание, мы перенесли сюда и главное меню, т.к. оно используется напрямую классом **DataMixin**.

Что же делает класс **DataMixin**? В нем объявлен вспомогательный метод **get_mixin_context()** для формирования контекста шаблона по умолчанию. Также, при необходимости, мы можем передавать ему именованные аргументы, которые также будут помещаться в контекст. Благодаря этому методу, нам не придется в классах представлений каждый раз прописывать ссылки на главное меню.

Теперь осталось в классе **WomenHome** изменить метод **get_context_data()**, следующим образом:

```
def get_context_data(self, *, object_list=None,
**kwargs):
    return
self.get_mixin_context(super().get_context_data(**kwargs),
                        title='Главная
страница',
                        cat_selected=0,
                        )
```

Мы здесь вызываем метод **get_mixin_context()** класса **DataMixin**, указав дополнительно параметры **title** и **cat_selected**. Получаем новый словарь со всеми стандартными ключами и объединяем его со словарем **context**. После этого он возвращается.

По аналогии, меняем и все остальные классы представлений, где используется вызов **get_context_data()**:

```
class ShowPost(DataMixin, DetailView):
...

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        return self.get_mixin_context(context,
title=context['post'])

class WomenCategory(DataMixin, ListView):
...

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        cat = context['posts'][0].cat
        return self.get_mixin_context(context,
                                     title='Категория
- ' + cat.name,
cat_selected=cat.id,
                                     )

class TagPostList(DataMixin, ListView):
...
```

```

def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    tag =
TagsPost.objects.get(slug=self.kwargs['tag_slug'])
    return self.get_mixin_context(context,
title='Ter: ' + tag.tag)

```

Переходим на сайт и видим, что страницы отображаются так же, как и ранее, но теперь все работает совместно с классом **DataMixin**.

Можно пойти еще дальше и оптимизировать с помощью этого же класса **DataMixin** определение словаря **extra_context**. Для этого в классе **DataMixin** мы пропишем следующие строки:

```

class DataMixin:
    title_page = None
    extra_context = {}

    def __init__(self):
        if self.title_page:
            self.extra_context['title'] =
self.title_page

        if 'menu' not in self.extra_context:
            self.extra_context['menu'] = menu

    def get_mixin_context(self, context, **kwargs):
        if self.title_page:
            context['title'] = self.title_page

        context['menu'] = menu
        context['cat_selected'] = None
        context.update(kwargs)
        return context

```

Здесь вводится специальный атрибут **title_page**, который может содержать заголовок страницы, а также пустой словарь **extra_context**. Ниже записан инициализатор для формирования в словаре **extra_context** ключа **menu** и ключа **title**, если атрибут **title_page** не равен **None**, то есть, определен в дочернем классе. Перейдем теперь в класс **AddPage** и модифицируем его следующим образом:

```

class AddPage(DataMixin, CreateView):
    model = Women

```

```

fields = ['title', 'slug', 'content',
'is_published', 'cat']
# form_class = AddPostForm
template_name = 'women/addpage.html'
success_url = reverse_lazy('home')
title_page = 'Добавление статьи'

```

Мы теперь можем пользоваться атрибутом **title_page**, введенным в классе **DataMixin**. Содержимое этого атрибута автоматически добавится в словарь **extra_context** и будет передано в шаблон через параметр **title**. Все стало удобнее и нагляднее. При этом атрибут **extra_context**, по-прежнему, можно прописывать в классе **AddPage** и добавлять любые дополнительные данные в шаблон, то есть мы нашим классом **DataMixin** никак не нарушаем стандартную логику работы фреймворка Django.

Внесем аналогичные изменения в другие классы, которые используют словарь **extra_context**:

```

class UpdatePage(DataMixin, UpdateView):
...
title_page = 'Редактирование статьи'

```

Вот пример того, как миксины в Django позволяют устранять дублирование кода и добавлять новый функционал в классы представления.

Введение в пагинацию. Класс Paginator

Сделаем следующее улучшение нашего сайта – постраничную навигацию. Иногда еще говорят «пагинация» (от лат. *pagina* – страница). Но это все о том, как на сайте представлять длинные списки чего-либо. Вот классический пример постраничной навигации:

7

Показания счётчика электроэнергии 1 ноября составляли 12 625 кВт·ч, а 1 декабря — 12 802 кВт·ч. Сколько нужно заплатить за электроэнергию за ноябрь, если 1 кВт·ч электроэнергии стоит 1 рубль 80 копеек? Ответ дайте в рублях.

Решение

8

Маша отправила SMS-сообщения с новогодними поздравлениями своим 16 друзьям. Стоимость одного SMS-сообщения 1 рубль 30 копеек. Перед отправкой сообщения на счету у Маши было 30 рублей. Сколько рублей останется у Маши после отправки всех сообщений?

Решение

9

Поезд Новосибирск-Красноярск отправляется в 15:20, а прибывает в 4:20 на следующий день (время московское). Сколько часов поезд находится в пути?

Решение

10

Спидометр автомобиля показывает скорость в милях в час. Какую скорость (в милях в час) показывает спидометр, если автомобиль движется со скоростью 36 км в час? (Считайте, что 1 миля равна 1,6 км.)

Решение

1

2

3

4

5

Здесь длинный список разбивается на блоки по 10 задач и внизу следуют номера страниц для отображения остальных номеров. Это рекомендуемая практика. Так HTML-страница получается не слишком большой по объему и пользователю удобнее ориентироваться в данных списка.

Если на странице официальной документации

<https://docs.djangoproject.com/en/4.2/>

набрать запрос «pagination», то сразу увидим первые две подходящие ссылки. По первой ссылке представлен пример работы класса пагинатора:

<https://docs.djangoproject.com/en/4.2/topics/pagination/>

А по второй – API данного класса (набор методов и свойств для работы с ним):

<https://docs.djangoproject.com/en/4.2/ref/paginator/>

Рассмотрим пример работы класса **Paginator**. Предположим, имеется следующий список имен известных женщин:

```
women = ['Анджелина Джоли', 'Дженнифер Лоуренс',  
'Джулия Робертс', 'Марго Робби', 'Ума Турман', 'Ариана Гранде',  
'Бейонсе', 'Кэтти Перри', 'Рианна', 'Шакира']
```

И нам нужно выводить их постранично, допустим, по три имени на странице. Для этого сначала выполним импорт класса:

```
from django.core.paginator import Paginator
```

и создадим его экземпляр:

```
p = Paginator(women, 3)
```

Наш пагинатор готов к использованию. Например:

```
p.count # число элементов в списке  
p.num_pages # число страниц (10:3 = 4 - округление до  
большого)  
p.page_range # итератор для перебора номеров страниц
```

Чтобы получить первую страницу со списком имен, используется метод:

```
p1 = p.page(1) # получение первой страницы  
p1.object_list # список элементов текущей страницы  
p1.has_next() # имеется ли следующая страница
```



```
p1.has_previous() # имеется ли предыдущая страница
p1.has_other_pages() # имеются ли вообще страницы
p1.next_page_number() # номер следующей страницы
p1.previous_page_number() # номер предыдущей страницы
```

Мы не будем приводить здесь полный список свойств и методов объекта пагинации, все это можно посмотреть на странице документации по указанным ссылкам.

Посмотрим, как все это можно использовать непосредственно на сайте. Если применяются функции представления, то нужно самостоятельно создавать объект класса **Paginator**, получать номер текущей страницы и для нее формировать объект с набором элементов. В документации это все приведено, а здесь мы рассмотрим пример, как можно этим воспользоваться в рамках нашего сайта.

У нас есть функция представления **about()**. Добавим в нее следующие строки:

```
contact_list = Women.published.all()
paginator = Paginator(contact_list, 3)

page_number = request.GET.get('page')
page_obj = paginator.get_page(page_number)
```

Мы прописали нашу модель **Women** и указали отображать 3 элемента на странице. Коллекцию **page_obj** для текущей страницы нужно передать в шаблон:

```
return render(request, 'women/about.html', {'page_obj':
page_obj, 'title': 'О сайте'})
```

А в самом шаблоне **about.html**, перебрать эту коллекцию и отобразить на странице:

```
{% for contact in page_obj %}
{{ contact }}</p>
{% endfor %}
```

Если теперь открыть страницу «О сайте», то увидим первые три имени из модели **Women**. Чтобы перейти на следующую страницу, нужно в запросе указать специальный параметр **page** с номером страницы, например, так:

<http://127.0.0.1:8000/about/?page=2>

Именно этот параметр читается из словаря **GET** и подставляется в метод **get_page** для получения соответствующей страницы. Причем при указании несуществующего номера будет отображаться последняя страница. Если вместо номера указать, например, буквы, то параметр игнорируется и показываются первые три записи.

Осталось отобразить ссылки на номера страниц непосредственно в шаблоне. Для этого мы через объект **page_obj** обратимся к объекту **paginator** и у него укажем объект-свойство **page_range**:

`page_obj.paginator.page_range`

Обратите внимание, в шаблонах у методов мы не пишем круглые скобки, весь вызов берет на себя шаблонизатор. Итак, **page_range** возвращает итератор для перебора номеров страниц. Поэтому в шаблоне мы можем воспользоваться тегом **for** для формирования номеров страниц:

```
<nav>
  <ul>
    {% for p in page_obj.paginator.page_range %}
    <li>
      <a href="?page={{ p }}">{{ p }}</a>
    </li>
    {% endfor %}
  </ul>
</nav>
```

Здесь мы перебираем доступные номера, формируем ссылку с параметром **page** и текущим номером. Именем ссылки также выступает ее номер. Если теперь перейти в браузер и обновить страницу, то увидим номера страниц.

Пагинация с классом **ListView**

Добавим пагинацию в класс представления **ListView**. В нем уже встроен механизм постраничного вывода списков. Согласно документации все, что нам нужно, это определить атрибут:

`paginate_by = N`

указав число **N** – количество элементов на одной странице. Например, если в классе **WomenHome** прописать этот атрибут:

```
class WomenHome(DataMixin, ListView):
    template_name = 'women/index.html'
    context_object_name = 'posts'
```

```
paginate_by = 3
```

то получим разбивку по три поста на страницу. Если теперь открыть главную страницу, то именно первые три записи мы и увидим. Но у нас пока нет отображения номеров страниц. Давайте добавим их. Для этого в шаблоне **base.html** добавим блок навигации после блока **content**:

```
{% block navigation %}
{% endblock %}
```

Затем в шаблоне **index.html** пропишем содержимое этого блока следующим образом:

```
<nav class="list-pages">
  <ul>
    {% for p in paginator.page_range %}
    <li class="page-num">
      <a href="?page={{ p }}">{{ p }}</a>
    </li>
    {% endfor %}
  </ul>
</nav>
```

Обратите внимание, на уровне шаблонов у нас появляются специальные объекты **paginator** и **page_obj**. В частности, с помощью **paginator** мы формируем отображение ссылок на страницы.

Если теперь обновить главную страницу сайта, то увидим номера страниц в виде списка вместе со стилями, которые подключены через классы оформлений **list-pages** и **page-num**. Эти стили определены в файле **styles.css**.

Главная страница отображается в нужном нам виде, но, если открыть отдельные рубрики, то пагинация пропадает. Почему? В проекте за это отвечает отдельный класс **WomenCategory**, в котором, по идее, также нужно прописать

```
paginate_by = 3
```

Однако, это будет дублирование кода, и нам проще эту строку записать в общем классе **DataMixin**:

```
class DataMixin:
    paginate_by = 3
    ...
```

А из класса **WomenHome** ее убрать. Теперь на всех страницах, унаследованных от **ListView**, мы будем видеть автоматическую пагинацию. Правда, везде будет отображаться по три статьи. В данном случае нас это устраивает, но, если на разных страницах нужно разное число рубрик при пагинации, то, в каждом классе отдельно следует прописать атрибут **paginate_by** со своим значением.

Если сейчас в **paginate_by** указать большое число, например, 30, то на главной странице будет отображаться внизу один единственный номер, что не очень хорошо. Было бы лучше, при одной единственной странице вообще не выводить никаких номеров. Для этого в шаблоне **index.html** вывод пагинации будем осуществлять по следующему условию:

```
{% if page_obj.has_other_pages %}
<nav class="list-pages">
...
</nav>
{% endif %}
```

Теперь базовый вариант отображения номеров страниц у нас есть. Сделаем еще некоторые улучшения. Например, уберем ссылку на текущий номер страницы. Для этого в шаблоне **index.html** мы, при выводе ссылок, будем проверять, если это текущий номер, то отображать его как текст:

```
<nav class="list-pages">
  <ul>
    {% for p in paginator.page_range %}
      {% if page_obj.number == p %}
        <li class="page-num page-num-selected">{{ p }}</li>
      {% else %}
        <li class="page-num">
          <a href="?page={{ p }}">{{ p }}</a>
        </li>
      {% endif %}
    {% endfor %}
  </ul>
</nav>
```

Здесь используется свойство **number** объекта **page_obj** для определения номера текущей страницы, и если этот номер равен **p**, то его следует отобразить как текст. Иначе, идет отображение в виде ссылки.

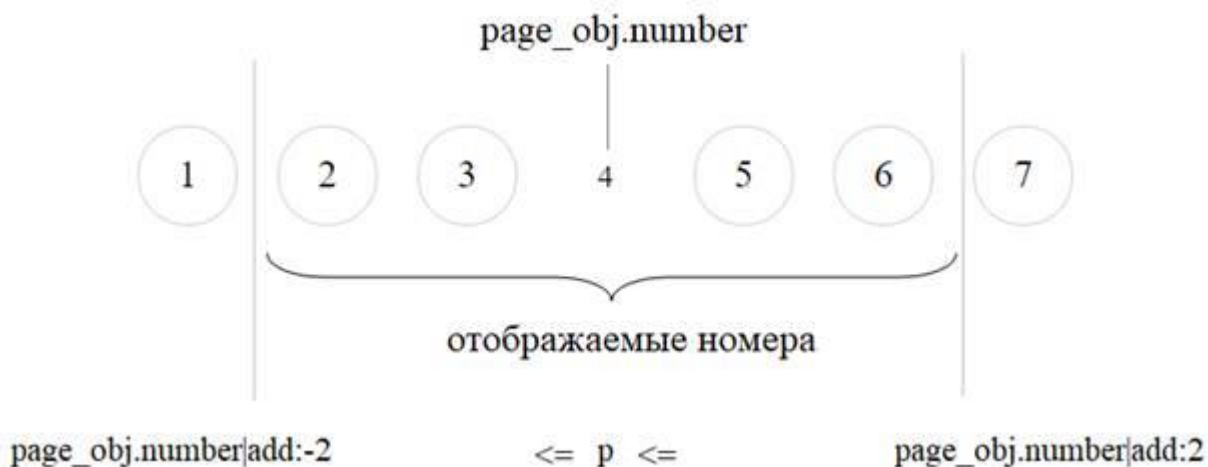
Следующая проблема, с которой мы можем столкнуться – это большое число номеров при большом числе записей. И при навигации это будет приводить к некоторым неудобствам, да и на дизайне страницы большое

число номеров сказывается не лучшим образом. Поэтому на практике, ограничиваются небольшим числом отображаемых номеров слева и справа от текущего номера. Вот это мы сейчас и сделаем.

Для начала в админ-панели отметим все статьи для публикации (чтобы номеров страниц было больше) и в классе **DataMixin** установим атрибут:

```
paginate_by = 2
```

Теперь на главной странице у нас появляется семь номеров. Но мы решили отображать не все их, а, допустим, два справа и два слева:



Этот диапазон, на уровне шаблонов, можно определить через фильтр **add**, уменьшая значение на 2 и увеличивая на 2 относительно текущего номера страницы. Поэтому в цикле, при отображении этих номеров, мы можем записать следующее условие вывода:

```
{% for p in paginator.page_range %}
    {% if page_obj.number == p %}
    <li class="page-num page-num-selected">{{ p }}</li>
    {% elif p >= page_obj.number|add:-2
and p <= page_obj.number|add:2 %}
    <li class="page-num">
        <a href="?page={{ p }}">{{ p }}</a>
    </li>
    {% endif %}
{% endfor %}
```

Теперь, обновляя страницу сайта, мы увидим по два номера слева и справа.

Следующим нашим улучшением будет отображение ссылок для перехода на предыдущую и следующую страницы. В шаблоне **index.html** до цикла запишем такие строки:

```
{% if page_obj.has_previous %}
<li class="page-num">
    <a href="?page={{
page_obj.previous_page_number }}">&lt;</a>
</li>
{% endif %}
```

Мы здесь сначала проверяем, имеет ли смысл вообще выводить ссылку на предыдущую страницу (для первых номеров это не нужно делать). И, если она нужна, то отображаем как элемент списка с номером страницы согласно атрибуту **previous_page_number**.

То же самое делаем и для отображения ссылки на следующую страницу. Только теперь добавляем пункт после цикла **for**:

```
{% if page_obj.has_next %}
<li class="page-num">
    <a href="?page={{ page_obj.next_page_number
 }}">&gt;</a>
</li>
{% endif %}
```

Здесь также сначала идет проверка: нужно ли отображать ссылку. Если проверка проходит, то отображаем пункт списка со ссылкой согласно атрибуту **next_page_number**.

Обновим главную страницу сайта и увидим эти дополнительные элементы управления.

Задание

1. Выполнить перепроектирование (рефакторинг) исходного кода, заменив функции представления классами представлений. Использовать классы View, TemplateView, ListView, DetailView, FormView, CreateView, UpdateView и, предварительно изучив по документации, DeleteView.
2. Вынести общий код классов представлений в отдельный класс-миксин. Используя этот класс, оптимизировать код, как это описано в методичке.
3. Добавить на сайт постраничную навигацию. При этом обеспечить следующие возможности:
 - вывод номеров страниц;

- отсутствие вывода номеров страниц в случае, если количество записей на странице совпадает или превышает общее количество записей в БД;
- отсутствие ссылки на текущий номер страницы (отображается как текст);
- ограничение небольшим числом отображаемых номеров слева и справа от текущего номера;
- отображение ссылок для перехода на предыдущую и следующую страницы с проверкой необходимости отображения этих ссылок.

Для того, чтобы была возможность в полной мере проверить, а, в дальнейшем, и продемонстрировать работу постраничной навигации, при необходимости добавьте в БД достаточное количество записей.